Mini Project Report

ML-Powered Medical Diagnosis Assistant

Final Version - 14 June 2026

Submitted by

Roll No	Name
24BDS062	Rachith Bharadwaj T N
24BDS018	Dhanush Gowda N
24BDS076	Shreedhar M Kadkol
24BDS031	Kishan Kumar Y

Under the guidance of

Dr. Utkarsh Mahadeo Khaire

Department of Data Science and Artificial Intelligence

Indian Institute of Information Technology Dharwad

GitHub Repository: https://github.com/Rachith000bharadwaj/Machine-Learning

Certificate

This is to certify that the mini project entitled **ML-Powered Medical Diagnosis Assistant** is a bonafide record of project work carried out by the students listed below. The project implements an educational symptom-based machine learning diagnosis assistant using a trained TensorFlow classifier and a Flask web interface.

Roll No	Name
24BDS062	Rachith Bharadwaj T N
24BDS018	Dhanush Gowda N
24BDS076	Shreedhar M Kadkol
24BDS031	Kishan Kumar Y

Project Supervisor: Dr. Utkarsh Mahadeo Khaire

Department of DSAI, IIIT Dharwad

Abstract

The ML-Powered Medical Diagnosis Assistant is an educational prototype that predicts a preliminary disease label from patient-reported symptoms. The final runnable system includes a Flask web app, a saved TensorFlow neural network model, a label encoder, symptom vocabulary metadata, ranked predictions, confidence scores, matched symptom evidence, urgency labels, recommended actions, voice input support through the browser, and printable diagnosis reports.

The final implementation is intentionally presented as a classroom machine learning prototype, not a medical device. Larger ideas such as Apache Spark EHR validation, native Android deployment, clinical data integration, and encryption are retained as future work rather than claimed as completed features. This improves credibility and keeps the project aligned with the code that can be run from GitHub.

After final cleaning, the dataset contains 488 training rows, 123 testing rows, 42 disease classes, and 137 binary symptom features. The saved model achieved 99.49 percent training accuracy, 98.98 percent validation accuracy, and 100 percent accuracy, precision, recall, and F1-score on the included clean classroom test file. These results should be interpreted as project-dataset performance, not real-world clinical accuracy.

Contents

1. Introduction

Healthcare triage often begins with symptom descriptions. In many rural and resource-limited settings, patients may not have immediate access to qualified medical professionals, and basic decision support can help organize symptoms before consultation. This project explores how a supervised machine learning model can map symptom combinations to likely disease labels in an educational setting.

The project focuses on a practical, offline-friendly web demonstration. A user enters symptoms in text or uses browser voice input. The app maps recognized symptom phrases to binary model features and returns a ranked diagnosis report with confidence, urgency, matched symptoms, and suggested next action.

1.1 Objectives

- Build a working symptom-to-disease classifier from project symptom datasets.

- Provide a simple Flask web interface that can run locally without ngrok.

- Show confidence scores, ranked alternatives, matched symptoms, and urgency guidance.

- Keep the repository reproducible with setup instructions, scripts, model artifacts, and documentation.

- State safety limitations clearly so the project is not confused with a clinical product.

2. Problem Statement

Patients frequently describe symptoms in natural language, while machine learning models require structured features. The project problem is to convert simple symptom descriptions into a binary feature vector, use a trained classifier to predict likely diseases, and present the result in a way that is understandable to a non-technical user.

The system must also remain honest about its boundaries: it cannot use laboratory values, imaging, vital signs, doctor notes, or regulated clinical validation. Therefore, its role is educational preliminary triage support, not diagnosis replacement.

3. Proposed Solution

The final project implements a local web-based diagnosis assistant. The backend loads a TensorFlow model, a scikit-learn label encoder, and a JSON symptom vocabulary. The frontend accepts user symptoms, sends them to the Flask API, and displays the model output as a printable report.

Component	Final implementation
Input	Text symptoms and browser-supported voice input
Feature mapping	Exact phrase and alias matching against 137 symptom features
Model	TensorFlow neural network saved as BACKEND/models/ml_model.h5
Output	Primary diagnosis, top 3 predictions, confidence, urgency, action, matched evidence
Deployment	Local Flask app with offline/local-network run script

Spark EHR validation, Android deployment, encryption, and large clinical-dataset validation are planned extensions. They are not required to run the current GitHub demo.

4. System Architecture

The architecture is intentionally compact so that it can be understood, run, and evaluated from the project folder. The model artifacts are kept under BACKEND/models, the Flask app is under WEB-APP, dataset cleaning and model scripts are under scripts, and documentation is kept at the repository root and docs

folder.

Layer	Responsibility	Files
Dataset layer	Stores original and cleaned symptom datasets	DATASETS/
Preprocessing layer	Cleans labels, removes duplicates, ignores placeholder symptoms	scripts/clean_datasets.py
Model layer	Trains and saves TensorFlow model, encoder, and vocabulary	scripts/train_model.py, BACKEND/models/
API layer	Loads model, matches symptoms, serves diagnosis endpoint	WEB-APP/app.py
Interface layer	Captures symptoms and renders diagnosis report	WEB-APP/templates, WEB-APP/static
Documentation layer	Explains setup, limitations, metrics, upload steps	README.md, MODEL_CARD.md, docs/

5. Dataset and Preprocessing

The project uses symptom-to-disease CSV files available in the DATASETS folder. Cleaning is project-focused: it combines app-compatible disease-symptom sources, normalizes disease names, normalizes symptom names, removes duplicate disease-symptom records, removes classes with too few examples, and writes cleaned training and testing CSV files.

Source	Rows loaded by cleaner
Training.csv	306
Testing.csv	42
dataset.csv	4920
dataset (1).csv	4925
cleaned_dataset.csv	309

Final cleaned output	Value
Clean records before final split	10502 before de-duplication and filtering
Training rows	488
Testing rows	123
Disease classes	42
Binary symptom features	137
Important final cleanup	Placeholder symptom values such as 0 are ignored

The cleaned dataset is saved to DATASETS/cleaned/Training_cleaned.csv and DATASETS/cleaned/Testing_cleaned.csv. A JSON summary is saved as DATASETS/cleaned/cleaning_summary.json.

6. Model Development

The final saved classifier is a TensorFlow neural network trained on binary symptom features. A scikit-learn LabelEncoder maps disease names to numeric classes during training and back to disease names during

inference. The symptom vocabulary is saved separately as JSON so the web app can construct input vectors in the exact same feature order used during training.

Model detail	Value
Input dimension	137 binary symptom features
Hidden layers	Dense 128 ReLU, dropout 0.25, Dense 64 ReLU, dropout 0.15
Output layer	42-class softmax
Optimizer	Adam, learning rate 0.001
Loss	Sparse categorical cross-entropy
Training split	390 training samples and 98 validation samples from Training_cleaned.csv
Final epoch	22

The model is saved as an HDF5 file for compatibility with the current Flask app. Future work may migrate this artifact to the native Keras format.

7. Web Application Implementation

The web application is implemented with Flask, HTML, CSS, and JavaScript. It exposes a health endpoint, a diagnosis endpoint, and a feedback endpoint. The interface uses local static files so it can run without internet-loaded page assets. A Windows batch file starts the app in offline/local-network mode.

Feature	How it works
Model health	GET /api/health returns readiness and model status
Diagnosis	POST /diagnose receives symptom text and returns JSON report fields
Voice input	Uses browser SpeechRecognition when supported
Urgency	Rule-based labels use high-risk symptoms, disease terms, and confidence
Printable report	Browser print view hides input/navigation and prints the diagnosis panel
Local logs	Diagnosis and feedback JSONL files are written under WEB-APP/logs and ignored by Git

8. Evaluation and Results

The final project was checked by running the dataset cleaner, retraining the model, and evaluating the saved model against the cleaned test file. The evaluation script uses the same saved model, label encoder, and vocabulary that the Flask app loads.

Metric	Final value
Training accuracy	0.9949
Validation accuracy	0.9898
Clean test samples	123
Clean test classes	42
Clean test accuracy	1.0000

Metric	Final value
Clean test precision	1.0000
Clean test recall	1.0000
Clean test F1-score	1.0000

The perfect clean-test score is useful for classroom verification, but it does not prove clinical accuracy. The test file is small, structured, and drawn from project-style symptom data rather than messy real patient records.

Example symptoms tested through the app include respiratory symptoms such as high fever, cough, chest pain, chills, fatigue, phlegm, and breathlessness, as well as headache/migraine-like symptom combinations.

9. Safety, Ethics, and Limitations

Medical machine learning systems require careful communication. This project includes a disclaimer in the README, model card, and web interface. The output should be treated as educational decision support only.

- The model does not use lab tests, imaging, vitals, patient history, doctor notes, or physical examination data.

- Confidence scores are softmax probabilities, not calibrated medical certainty.

- The app depends on matching user text to the known symptom vocabulary; vague or unusual wording can reduce quality.

- Urgency labels are simple rules and are not a substitute for emergency triage protocols.

- The model has not been externally validated on hospital EHR data or real clinical workflows.

10. GitHub Reproducibility

The project repository is prepared for GitHub at https://github.com/Rachith000bharadwaj/Machine-Learning. The README explains setup, local running, evaluation, dataset cleaning, model retraining, and the difference between completed features and planned extensions.

Reproducibility item	Location
Setup requirements	requirements.txt
Run app offline/local	run_offline.bat or python WEB-APP/app.py
Clean dataset	python scripts/clean_datasets.py
Retrain model	python scripts/train_model.py
Evaluate saved model	python scripts/evaluate_model.py
Model documentation	MODEL_CARD.md
Upload notes	docs/GITHUB_UPLOAD_STEPS.md

11. Future Enhancements

The final project leaves several strong future directions. These should be presented as planned work unless they are implemented and validated in a later version.

- Integrate larger and more diverse clinical datasets after proper access, privacy, and ethics review.

- Add Spark-based batch validation for EHR-style records as a separate validated module.

- Build a native Android app or package the Flask app for offline field demonstrations.

- Improve natural-language symptom extraction beyond simple phrase matching.

- Calibrate confidence scores and evaluate robustness on noisy user-entered symptoms.

- Add multilingual symptom support for Indian languages.

12. Conclusion

The ML-Powered Medical Diagnosis Assistant now has a clear final form: a runnable, documented, GitHub-ready educational prototype. It cleans project symptom datasets, trains a TensorFlow classifier, serves predictions through Flask, and presents diagnosis reports with confidence, alternatives, urgency, and matched evidence.

The strongest part of the final version is its alignment between code, report, README, model card, and safety language. Instead of overclaiming advanced clinical infrastructure, it demonstrates a focused machine learning workflow that can be installed, evaluated, and discussed honestly.

References

TensorFlow documentation, https://www.tensorflow.org/

scikit-learn documentation, https://scikit-learn.org/

Flask documentation, https://flask.palletsprojects.com/

Pandas documentation, https://pandas.pydata.org/

Project symptom-to-disease CSV files included under the DATASETS folder.

Appendix: Quick Start Commands

Task	Command
Create environment	python -m venv .venv
Activate on Windows	.venv\Scripts\activate
Install dependencies	pip install -r requirements.txt
Run app	cd WEB-APP && python app.py
Clean data	python scripts\clean_datasets.py
Train model	python scripts\train_model.py
Evaluate model	python scripts\evaluate_model.py